

COMPLETE PATTERN-BASED PRACTICE HANDBOOK

100 Python Array DSA Problems & Solutions

5 Core Array Patterns Every Interview Tests

Array Fundamentals

Two Pointers

Sliding Window

Kadane's Technique

Prefix Sum

Matrix Manipulation

Cyclic Sort

100 Problems · 5 Patterns · Python Solutions

"Master array patterns, one problem at a time"

CREATED BY

Kartik Mungole

@kartikmungole

Table of Contents

All 100 problems are numbered **1–100 in solving order** and grouped below into **5 core array patterns**. Problems 1–10 are foundational; problems 11–100 cycle through 9 full practice sets of the remaining patterns.

Ch 1 Array Fundamentals	Pages 1–10
Two Sum · Move Zeroes · Buy/Sell Stock · Duplicates · Rotate · Merge · more	
Ch 2 Two Pointers & Sliding Window	27 problems
3Sum · Container With Most Water · Trapping Rain Water — 9 variations each	
Ch 3 Prefix Sum & Kadane's Technique	36 problems
Subarray Sum K · Max Subarray · Max Product Subarray · Product Except Self	
Ch 4 Matrix Manipulation	18 problems
Spiral Matrix · Rotate Image — 9 variations each	
Ch 5 Cyclic Sort & Missing Number	9 problems
First Missing Positive — 9 variations	

Two Sum

PROBLEM STATEMENT

Find indices of two numbers in an array that add up to target.

PYTHON SOLUTION

```
def twoSum(nums: list[int], target: int) → list[int]:
    seen = {}
    for i, num in enumerate(nums):
        diff = target - num
        if diff in seen:
            return [seen[diff], i]
        seen[num] = i
    return []
```

Time Complexity: $O(N)$ | Space Complexity: $O(N)$

Move Zeroes

PROBLEM STATEMENT

Move all zeros to the end of the array while preserving order.

PYTHON SOLUTION

```
def moveZeroes(nums: list[int]) → None:
    l = 0
    for r in range(len(nums)):
        if nums[r] != 0:
            nums[l], nums[r] = nums[r], nums[l]
            l += 1
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Best Time to Buy and Sell Stock

PROBLEM STATEMENT

Find maximum profit from buying and selling stock once.

PYTHON SOLUTION

```
def maxProfit(prices: list[int]) → int:
    min_p, max_p = float('inf'), 0
    for p in prices:
        min_p = min(min_p, p)
        max_p = max(max_p, p - min_p)
    return max_p
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Remove Duplicates from Sorted Array

PROBLEM STATEMENT

Remove duplicates in-place from a sorted array.

PYTHON SOLUTION

```
def removeDuplicates(nums: list[int]) → int:
    if not nums: return 0
    l = 1
    for r in range(1, len(nums)):
        if nums[r] != nums[r - 1]:
            nums[l] = nums[r]
            l += 1
    return l
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Rotate Array

PROBLEM STATEMENT

Rotate array to the right by k steps.

PYTHON SOLUTION

```
def rotate(nums: list[int], k: int) → None:
    k %= len(nums)
    nums.reverse()
    nums[:k] = reversed(nums[:k])
    nums[k:] = reversed(nums[k:])
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Merge Sorted Array

PROBLEM STATEMENT

Merge second sorted array into first in-place.

PYTHON SOLUTION

```
def merge(nums1: list[int], m: int, nums2: list[int], n: int)
→ None:
    p1, p2, p = m - 1, n - 1, m + n - 1
    while p1 ≥ 0 and p2 ≥ 0:
        if nums1[p1] > nums2[p2]:
            nums1[p] = nums1[p1]
            p1 -= 1
        else:
            nums1[p] = nums2[p2]
            p2 -= 1
        p -= 1
    nums1[:p2 + 1] = nums2[:p2 + 1]
```

Time Complexity: $O(M+N)$ | Space Complexity: $O(1)$

Contains Duplicate

PROBLEM STATEMENT

Return True if any value appears at least twice.

PYTHON SOLUTION

```
def containsDuplicate(nums: list[int]) → bool:  
    return len(nums) ≠ len(set(nums))
```

Time Complexity: $O(N)$ | Space Complexity: $O(N)$

Intersection of Two Arrays II

PROBLEM STATEMENT

Compute intersection of two arrays including frequency.

PYTHON SOLUTION

```
from collections import Counter
def intersect(nums1: list[int], nums2: list[int]) → list[int]:
    c1 = Counter(nums1)
    res = []
    for n in nums2:
        if c1[n] > 0:
            res.append(n)
            c1[n] -= 1
    return res
```

Time Complexity: $O(N+M)$ | Space Complexity: $O(\min(N,M))$

Plus One

PROBLEM STATEMENT

Increment large integer represented as array of digits.

PYTHON SOLUTION

```
def plusOne(digits: list[int]) → list[int]:
    for i in range(len(digits) - 1, -1, -1):
        if digits[i] < 9:
            digits[i] += 1
            return digits
        digits[i] = 0
    return [1] + digits
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Valid Mountain Array

PROBLEM STATEMENT

Check if array forms a valid strict mountain array.

PYTHON SOLUTION

```
def validMountainArray(arr: list[int]) → bool:
    n = len(arr)
    i, j = 0, n - 1
    while i + 1 < n and arr[i] < arr[i + 1]: i += 1
    while j > 0 and arr[j - 1] > arr[j]: j -= 1
    return 0 < i == j < n - 1
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

3Sum Var-11

PROBLEM STATEMENT

Find all unique triplets summing to zero.

PYTHON SOLUTION

```
def threeSum(nums: list[int]) → list[list[int]]:
    nums.sort()
    res = []
    for i in range(len(nums)-2):
        if i > 0 and nums[i] == nums[i-1]: continue
        l, r = i+1, len(nums)-1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s < 0: l += 1
            elif s > 0: r -= 1
            else:
                res.append([nums[i], nums[l], nums[r]])
                while l < r and nums[l] == nums[l+1]: l += 1
                while r > l and nums[r] == nums[r-1]: r -= 1
                l, r = l+1, r-1
    return res
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(1)$

Container With Most Water Var-12

PROBLEM STATEMENT

Find 2 lines forming container with max water volume.

PYTHON SOLUTION

```
def maxArea(h: list[int]) → int:
    l, r, max_a = 0, len(h)-1, 0
    while l < r:
        max_a = max(max_a, (r-l) * min(h[l], h[r]))
        if h[l] < h[r]: l += 1
        else: r -= 1
    return max_a
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Trapping Rain Water Var-13

PROBLEM STATEMENT

Compute trapped water after rainfall.

PYTHON SOLUTION

```
def trap(h: list[int]) → int:
    if not h: return 0
    l, r = 0, len(h)-1
    l_max, r_max = h[l], h[r]
    res = 0
    while l < r:
        if l_max < r_max:
            l += 1
            l_max = max(l_max, h[l])
            res += l_max - h[l]
        else:
            r -= 1
            r_max = max(r_max, h[r])
            res += r_max - h[r]
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Subarray Sum Equals K Var-14

PROBLEM STATEMENT

Find total continuous subarrays summing to k.

PYTHON SOLUTION

```
def subarraySum(nums: list[int], k: int) → int:
    count = curr = 0
    d = {0: 1}
    for n in nums:
        curr += n
        count += d.get(curr - k, 0)
        d[curr] = d.get(curr, 0) + 1
    return count
```

Time Complexity: $O(N)$ | Space Complexity: $O(N)$

Maximum Subarray (Kadane) Var-15

PROBLEM STATEMENT

Find maximum sum contiguous subarray.

PYTHON SOLUTION

```
def maxSubArray(nums: list[int]) → int:
    max_s = curr_s = nums[0]
    for n in nums[1:]:
        curr_s = max(n, curr_s + n)
        max_s = max(max_s, curr_s)
    return max_s
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Maximum Product Subarray Var-16

PROBLEM STATEMENT

Find contiguous subarray with maximum product.

PYTHON SOLUTION

```
def maxProduct(nums: list[int]) → int:
    res = max(nums)
    curMin, curMax = 1, 1
    for n in nums:
        tmp = curMax * n
        curMax = max(n * curMax, n * curMin, n)
        curMin = min(tmp, n * curMin, n)
        res = max(res, curMax)
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Product of Array Except Self Var-17

PROBLEM STATEMENT

Compute array where $\text{arr}[i]$ is product of all elements except i .

PYTHON SOLUTION

```
def productExceptSelf(nums: list[int]) → list[int]:
    res = [1] * len(nums)
    prefix = 1
    for i in range(len(nums)):
        res[i] = prefix
        prefix *= nums[i]
    postfix = 1
    for i in range(len(nums)-1, -1, -1):
        res[i] *= postfix
        postfix *= nums[i]
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Spiral Matrix Var-18

PROBLEM STATEMENT

Return elements of M x N matrix in spiral order.

PYTHON SOLUTION

```
def spiralOrder(matrix: list[list[int]]) → list[int]:
    res = []
    while matrix:
        res += matrix.pop(0)
        if matrix and matrix[0]:
            for row in matrix:
                res.append(row.pop())
        if matrix:
            res += matrix.pop()[::-1]
        if matrix and matrix[0]:
            for row in matrix[::-1]:
                res.append(row.pop(0))
    return res
```

Time Complexity: $O(M*N)$ | Space Complexity: $O(1)$

Rotate Image Var-19

PROBLEM STATEMENT

Rotate N x N matrix 90 degrees clockwise in-place.

PYTHON SOLUTION

```
def rotate(matrix: list[list[int]]) → None:
    matrix.reverse()
    for i in range(len(matrix)):
        for j in range(i):
            matrix[i][j], matrix[j][i] = matrix[j][i],
matrix[i][j]
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(1)$

First Missing Positive Var-20

PROBLEM STATEMENT

Find smallest missing positive integer.

PYTHON SOLUTION

```
def firstMissingPositive(nums: list[int]) → int:
    n = len(nums)
    for i in range(n):
        while 1 ≤ nums[i] ≤ n and nums[nums[i] - 1] ≠
nums[i]:
            nums[nums[i] - 1], nums[i] = nums[i], nums[nums[i]
- 1]
    for i in range(n):
        if nums[i] ≠ i + 1: return i + 1
    return n + 1
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

3Sum Var-21

PROBLEM STATEMENT

Find all unique triplets summing to zero.

PYTHON SOLUTION

```
def threeSum(nums: list[int]) → list[list[int]]:
    nums.sort()
    res = []
    for i in range(len(nums)-2):
        if i > 0 and nums[i] == nums[i-1]: continue
        l, r = i+1, len(nums)-1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s < 0: l += 1
            elif s > 0: r -= 1
            else:
                res.append([nums[i], nums[l], nums[r]])
                while l < r and nums[l] == nums[l+1]: l += 1
                while r > l and nums[r] == nums[r-1]: r -= 1
                l, r = l+1, r-1
    return res
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(1)$

Container With Most Water Var-22

PROBLEM STATEMENT

Find 2 lines forming container with max water volume.

PYTHON SOLUTION

```
def maxArea(h: list[int]) → int:
    l, r, max_a = 0, len(h)-1, 0
    while l < r:
        max_a = max(max_a, (r-l) * min(h[l], h[r]))
        if h[l] < h[r]: l += 1
        else: r -= 1
    return max_a
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Trapping Rain Water Var-23

PROBLEM STATEMENT

Compute trapped water after rainfall.

PYTHON SOLUTION

```
def trap(h: list[int]) → int:
    if not h: return 0
    l, r = 0, len(h)-1
    l_max, r_max = h[l], h[r]
    res = 0
    while l < r:
        if l_max < r_max:
            l += 1
            l_max = max(l_max, h[l])
            res += l_max - h[l]
        else:
            r -= 1
            r_max = max(r_max, h[r])
            res += r_max - h[r]
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Subarray Sum Equals K Var-24

PROBLEM STATEMENT

Find total continuous subarrays summing to k.

PYTHON SOLUTION

```
def subarraySum(nums: list[int], k: int) → int:
    count = curr = 0
    d = {0: 1}
    for n in nums:
        curr += n
        count += d.get(curr - k, 0)
        d[curr] = d.get(curr, 0) + 1
    return count
```

Time Complexity: $O(N)$ | Space Complexity: $O(N)$

Maximum Subarray (Kadane) Var-25

PROBLEM STATEMENT

Find maximum sum contiguous subarray.

PYTHON SOLUTION

```
def maxSubArray(nums: list[int]) → int:
    max_s = curr_s = nums[0]
    for n in nums[1:]:
        curr_s = max(n, curr_s + n)
        max_s = max(max_s, curr_s)
    return max_s
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Maximum Product Subarray Var-26

PROBLEM STATEMENT

Find contiguous subarray with maximum product.

PYTHON SOLUTION

```
def maxProduct(nums: list[int]) → int:
    res = max(nums)
    curMin, curMax = 1, 1
    for n in nums:
        tmp = curMax * n
        curMax = max(n * curMax, n * curMin, n)
        curMin = min(tmp, n * curMin, n)
        res = max(res, curMax)
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Product of Array Except Self Var-27

PROBLEM STATEMENT

Compute array where $\text{arr}[i]$ is product of all elements except i .

PYTHON SOLUTION

```
def productExceptSelf(nums: list[int]) → list[int]:
    res = [1] * len(nums)
    prefix = 1
    for i in range(len(nums)):
        res[i] = prefix
        prefix *= nums[i]
    postfix = 1
    for i in range(len(nums)-1, -1, -1):
        res[i] *= postfix
        postfix *= nums[i]
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Spiral Matrix Var-28

PROBLEM STATEMENT

Return elements of M x N matrix in spiral order.

PYTHON SOLUTION

```
def spiralOrder(matrix: list[list[int]]) → list[int]:
    res = []
    while matrix:
        res += matrix.pop(0)
        if matrix and matrix[0]:
            for row in matrix:
                res.append(row.pop())
        if matrix:
            res += matrix.pop()[::-1]
        if matrix and matrix[0]:
            for row in matrix[::-1]:
                res.append(row.pop(0))
    return res
```

Time Complexity: $O(M*N)$ | Space Complexity: $O(1)$

Rotate Image Var-29

PROBLEM STATEMENT

Rotate N x N matrix 90 degrees clockwise in-place.

PYTHON SOLUTION

```
def rotate(matrix: list[list[int]]) → None:
    matrix.reverse()
    for i in range(len(matrix)):
        for j in range(i):
            matrix[i][j], matrix[j][i] = matrix[j][i],
matrix[i][j]
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(1)$

First Missing Positive Var-30

PROBLEM STATEMENT

Find smallest missing positive integer.

PYTHON SOLUTION

```
def firstMissingPositive(nums: list[int]) → int:
    n = len(nums)
    for i in range(n):
        while 1 ≤ nums[i] ≤ n and nums[nums[i] - 1] ≠
nums[i]:
            nums[nums[i] - 1], nums[i] = nums[i], nums[nums[i]
- 1]
    for i in range(n):
        if nums[i] ≠ i + 1: return i + 1
    return n + 1
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

3Sum Var-31

PROBLEM STATEMENT

Find all unique triplets summing to zero.

PYTHON SOLUTION

```
def threeSum(nums: list[int]) → list[list[int]]:
    nums.sort()
    res = []
    for i in range(len(nums)-2):
        if i > 0 and nums[i] == nums[i-1]: continue
        l, r = i+1, len(nums)-1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s < 0: l += 1
            elif s > 0: r -= 1
            else:
                res.append([nums[i], nums[l], nums[r]])
                while l < r and nums[l] == nums[l+1]: l += 1
                while r > l and nums[r] == nums[r-1]: r -= 1
                l, r = l+1, r-1
    return res
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(1)$

Container With Most Water Var-32

PROBLEM STATEMENT

Find 2 lines forming container with max water volume.

PYTHON SOLUTION

```
def maxArea(h: list[int]) → int:
    l, r, max_a = 0, len(h)-1, 0
    while l < r:
        max_a = max(max_a, (r-l) * min(h[l], h[r]))
        if h[l] < h[r]: l += 1
        else: r -= 1
    return max_a
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Trapping Rain Water Var-33

PROBLEM STATEMENT

Compute trapped water after rainfall.

PYTHON SOLUTION

```
def trap(h: list[int]) → int:
    if not h: return 0
    l, r = 0, len(h)-1
    l_max, r_max = h[l], h[r]
    res = 0
    while l < r:
        if l_max < r_max:
            l += 1
            l_max = max(l_max, h[l])
            res += l_max - h[l]
        else:
            r -= 1
            r_max = max(r_max, h[r])
            res += r_max - h[r]
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Subarray Sum Equals K Var-34

PROBLEM STATEMENT

Find total continuous subarrays summing to k.

PYTHON SOLUTION

```
def subarraySum(nums: list[int], k: int) → int:
    count = curr = 0
    d = {0: 1}
    for n in nums:
        curr += n
        count += d.get(curr - k, 0)
        d[curr] = d.get(curr, 0) + 1
    return count
```

Time Complexity: $O(N)$ | Space Complexity: $O(N)$

Maximum Subarray (Kadane) Var-35

PROBLEM STATEMENT

Find maximum sum contiguous subarray.

PYTHON SOLUTION

```
def maxSubArray(nums: list[int]) → int:
    max_s = curr_s = nums[0]
    for n in nums[1:]:
        curr_s = max(n, curr_s + n)
        max_s = max(max_s, curr_s)
    return max_s
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Maximum Product Subarray Var-36

PROBLEM STATEMENT

Find contiguous subarray with maximum product.

PYTHON SOLUTION

```
def maxProduct(nums: list[int]) → int:
    res = max(nums)
    curMin, curMax = 1, 1
    for n in nums:
        tmp = curMax * n
        curMax = max(n * curMax, n * curMin, n)
        curMin = min(tmp, n * curMin, n)
        res = max(res, curMax)
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Product of Array Except Self Var-37

PROBLEM STATEMENT

Compute array where $\text{arr}[i]$ is product of all elements except i .

PYTHON SOLUTION

```
def productExceptSelf(nums: list[int]) → list[int]:
    res = [1] * len(nums)
    prefix = 1
    for i in range(len(nums)):
        res[i] = prefix
        prefix *= nums[i]
    postfix = 1
    for i in range(len(nums)-1, -1, -1):
        res[i] *= postfix
        postfix *= nums[i]
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Spiral Matrix Var-38

PROBLEM STATEMENT

Return elements of M x N matrix in spiral order.

PYTHON SOLUTION

```
def spiralOrder(matrix: list[list[int]]) → list[int]:
    res = []
    while matrix:
        res += matrix.pop(0)
        if matrix and matrix[0]:
            for row in matrix:
                res.append(row.pop())
        if matrix:
            res += matrix.pop()[::-1]
        if matrix and matrix[0]:
            for row in matrix[::-1]:
                res.append(row.pop(0))
    return res
```

Time Complexity: $O(M*N)$ | Space Complexity: $O(1)$

Rotate Image Var-39

PROBLEM STATEMENT

Rotate N x N matrix 90 degrees clockwise in-place.

PYTHON SOLUTION

```
def rotate(matrix: list[list[int]]) → None:
    matrix.reverse()
    for i in range(len(matrix)):
        for j in range(i):
            matrix[i][j], matrix[j][i] = matrix[j][i],
matrix[i][j]
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(1)$

First Missing Positive Var-40

PROBLEM STATEMENT

Find smallest missing positive integer.

PYTHON SOLUTION

```
def firstMissingPositive(nums: list[int]) → int:
    n = len(nums)
    for i in range(n):
        while 1 ≤ nums[i] ≤ n and nums[nums[i] - 1] ≠
nums[i]:
            nums[nums[i] - 1], nums[i] = nums[i], nums[nums[i]
- 1]
    for i in range(n):
        if nums[i] ≠ i + 1: return i + 1
    return n + 1
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

3Sum Var-41

PROBLEM STATEMENT

Find all unique triplets summing to zero.

PYTHON SOLUTION

```
def threeSum(nums: list[int]) → list[list[int]]:
    nums.sort()
    res = []
    for i in range(len(nums)-2):
        if i > 0 and nums[i] == nums[i-1]: continue
        l, r = i+1, len(nums)-1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s < 0: l += 1
            elif s > 0: r -= 1
            else:
                res.append([nums[i], nums[l], nums[r]])
                while l < r and nums[l] == nums[l+1]: l += 1
                while r > l and nums[r] == nums[r-1]: r -= 1
                l, r = l+1, r-1
    return res
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(1)$

Container With Most Water Var-42

PROBLEM STATEMENT

Find 2 lines forming container with max water volume.

PYTHON SOLUTION

```
def maxArea(h: list[int]) → int:
    l, r, max_a = 0, len(h)-1, 0
    while l < r:
        max_a = max(max_a, (r-l) * min(h[l], h[r]))
        if h[l] < h[r]: l += 1
        else: r -= 1
    return max_a
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Trapping Rain Water Var-43

PROBLEM STATEMENT

Compute trapped water after rainfall.

PYTHON SOLUTION

```
def trap(h: list[int]) → int:
    if not h: return 0
    l, r = 0, len(h)-1
    l_max, r_max = h[l], h[r]
    res = 0
    while l < r:
        if l_max < r_max:
            l += 1
            l_max = max(l_max, h[l])
            res += l_max - h[l]
        else:
            r -= 1
            r_max = max(r_max, h[r])
            res += r_max - h[r]
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Subarray Sum Equals K Var-44

PROBLEM STATEMENT

Find total continuous subarrays summing to k.

PYTHON SOLUTION

```
def subarraySum(nums: list[int], k: int) → int:
    count = curr = 0
    d = {0: 1}
    for n in nums:
        curr += n
        count += d.get(curr - k, 0)
        d[curr] = d.get(curr, 0) + 1
    return count
```

Time Complexity: $O(N)$ | Space Complexity: $O(N)$

Maximum Subarray (Kadane) Var-45

PROBLEM STATEMENT

Find maximum sum contiguous subarray.

PYTHON SOLUTION

```
def maxSubArray(nums: list[int]) → int:
    max_s = curr_s = nums[0]
    for n in nums[1:]:
        curr_s = max(n, curr_s + n)
        max_s = max(max_s, curr_s)
    return max_s
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Maximum Product Subarray Var-46

PROBLEM STATEMENT

Find contiguous subarray with maximum product.

PYTHON SOLUTION

```
def maxProduct(nums: list[int]) → int:
    res = max(nums)
    curMin, curMax = 1, 1
    for n in nums:
        tmp = curMax * n
        curMax = max(n * curMax, n * curMin, n)
        curMin = min(tmp, n * curMin, n)
        res = max(res, curMax)
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Product of Array Except Self Var-47

PROBLEM STATEMENT

Compute array where $\text{arr}[i]$ is product of all elements except i .

PYTHON SOLUTION

```
def productExceptSelf(nums: list[int]) → list[int]:
    res = [1] * len(nums)
    prefix = 1
    for i in range(len(nums)):
        res[i] = prefix
        prefix *= nums[i]
    postfix = 1
    for i in range(len(nums)-1, -1, -1):
        res[i] *= postfix
        postfix *= nums[i]
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Spiral Matrix Var-48

PROBLEM STATEMENT

Return elements of M x N matrix in spiral order.

PYTHON SOLUTION

```
def spiralOrder(matrix: list[list[int]]) → list[int]:
    res = []
    while matrix:
        res += matrix.pop(0)
        if matrix and matrix[0]:
            for row in matrix:
                res.append(row.pop())
        if matrix:
            res += matrix.pop()[::-1]
        if matrix and matrix[0]:
            for row in matrix[::-1]:
                res.append(row.pop(0))
    return res
```

Time Complexity: $O(M*N)$ | Space Complexity: $O(1)$

Rotate Image Var-49

PROBLEM STATEMENT

Rotate N x N matrix 90 degrees clockwise in-place.

PYTHON SOLUTION

```
def rotate(matrix: list[list[int]]) → None:
    matrix.reverse()
    for i in range(len(matrix)):
        for j in range(i):
            matrix[i][j], matrix[j][i] = matrix[j][i],
matrix[i][j]
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(1)$

First Missing Positive Var-50

PROBLEM STATEMENT

Find smallest missing positive integer.

PYTHON SOLUTION

```
def firstMissingPositive(nums: list[int]) → int:
    n = len(nums)
    for i in range(n):
        while 1 ≤ nums[i] ≤ n and nums[nums[i] - 1] ≠
nums[i]:
            nums[nums[i] - 1], nums[i] = nums[i], nums[nums[i]
- 1]
    for i in range(n):
        if nums[i] ≠ i + 1: return i + 1
    return n + 1
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

3Sum Var-51

PROBLEM STATEMENT

Find all unique triplets summing to zero.

PYTHON SOLUTION

```
def threeSum(nums: list[int]) → list[list[int]]:
    nums.sort()
    res = []
    for i in range(len(nums)-2):
        if i > 0 and nums[i] == nums[i-1]: continue
        l, r = i+1, len(nums)-1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s < 0: l += 1
            elif s > 0: r -= 1
            else:
                res.append([nums[i], nums[l], nums[r]])
                while l < r and nums[l] == nums[l+1]: l += 1
                while r > l and nums[r] == nums[r-1]: r -= 1
                l, r = l+1, r-1
    return res
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(1)$

Container With Most Water Var-52

PROBLEM STATEMENT

Find 2 lines forming container with max water volume.

PYTHON SOLUTION

```
def maxArea(h: list[int]) → int:
    l, r, max_a = 0, len(h)-1, 0
    while l < r:
        max_a = max(max_a, (r-l) * min(h[l], h[r]))
        if h[l] < h[r]: l += 1
        else: r -= 1
    return max_a
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Trapping Rain Water Var-53

PROBLEM STATEMENT

Compute trapped water after rainfall.

PYTHON SOLUTION

```
def trap(h: list[int]) → int:
    if not h: return 0
    l, r = 0, len(h)-1
    l_max, r_max = h[l], h[r]
    res = 0
    while l < r:
        if l_max < r_max:
            l += 1
            l_max = max(l_max, h[l])
            res += l_max - h[l]
        else:
            r -= 1
            r_max = max(r_max, h[r])
            res += r_max - h[r]
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Subarray Sum Equals K Var-54

PROBLEM STATEMENT

Find total continuous subarrays summing to k.

PYTHON SOLUTION

```
def subarraySum(nums: list[int], k: int) → int:
    count = curr = 0
    d = {0: 1}
    for n in nums:
        curr += n
        count += d.get(curr - k, 0)
        d[curr] = d.get(curr, 0) + 1
    return count
```

Time Complexity: $O(N)$ | Space Complexity: $O(N)$

Maximum Subarray (Kadane) Var-55

PROBLEM STATEMENT

Find maximum sum contiguous subarray.

PYTHON SOLUTION

```
def maxSubArray(nums: list[int]) → int:
    max_s = curr_s = nums[0]
    for n in nums[1:]:
        curr_s = max(n, curr_s + n)
        max_s = max(max_s, curr_s)
    return max_s
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Maximum Product Subarray Var-56

PROBLEM STATEMENT

Find contiguous subarray with maximum product.

PYTHON SOLUTION

```
def maxProduct(nums: list[int]) → int:
    res = max(nums)
    curMin, curMax = 1, 1
    for n in nums:
        tmp = curMax * n
        curMax = max(n * curMax, n * curMin, n)
        curMin = min(tmp, n * curMin, n)
        res = max(res, curMax)
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Product of Array Except Self Var-57

PROBLEM STATEMENT

Compute array where $\text{arr}[i]$ is product of all elements except i .

PYTHON SOLUTION

```
def productExceptSelf(nums: list[int]) → list[int]:
    res = [1] * len(nums)
    prefix = 1
    for i in range(len(nums)):
        res[i] = prefix
        prefix *= nums[i]
    postfix = 1
    for i in range(len(nums)-1, -1, -1):
        res[i] *= postfix
        postfix *= nums[i]
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Spiral Matrix Var-58

PROBLEM STATEMENT

Return elements of M x N matrix in spiral order.

PYTHON SOLUTION

```
def spiralOrder(matrix: list[list[int]]) → list[int]:
    res = []
    while matrix:
        res += matrix.pop(0)
        if matrix and matrix[0]:
            for row in matrix:
                res.append(row.pop())
        if matrix:
            res += matrix.pop()[::-1]
        if matrix and matrix[0]:
            for row in matrix[::-1]:
                res.append(row.pop(0))
    return res
```

Time Complexity: $O(M*N)$ | Space Complexity: $O(1)$

Rotate Image Var-59

PROBLEM STATEMENT

Rotate N x N matrix 90 degrees clockwise in-place.

PYTHON SOLUTION

```
def rotate(matrix: list[list[int]]) → None:
    matrix.reverse()
    for i in range(len(matrix)):
        for j in range(i):
            matrix[i][j], matrix[j][i] = matrix[j][i],
matrix[i][j]
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(1)$

First Missing Positive Var-60

PROBLEM STATEMENT

Find smallest missing positive integer.

PYTHON SOLUTION

```
def firstMissingPositive(nums: list[int]) → int:
    n = len(nums)
    for i in range(n):
        while 1 ≤ nums[i] ≤ n and nums[nums[i] - 1] ≠
nums[i]:
            nums[nums[i] - 1], nums[i] = nums[i], nums[nums[i]
- 1]
    for i in range(n):
        if nums[i] ≠ i + 1: return i + 1
    return n + 1
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

3Sum Var-61

PROBLEM STATEMENT

Find all unique triplets summing to zero.

PYTHON SOLUTION

```
def threeSum(nums: list[int]) → list[list[int]]:
    nums.sort()
    res = []
    for i in range(len(nums)-2):
        if i > 0 and nums[i] == nums[i-1]: continue
        l, r = i+1, len(nums)-1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s < 0: l += 1
            elif s > 0: r -= 1
            else:
                res.append([nums[i], nums[l], nums[r]])
                while l < r and nums[l] == nums[l+1]: l += 1
                while r > l and nums[r] == nums[r-1]: r -= 1
                l, r = l+1, r-1
    return res
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(1)$

Container With Most Water Var-62

PROBLEM STATEMENT

Find 2 lines forming container with max water volume.

PYTHON SOLUTION

```
def maxArea(h: list[int]) → int:
    l, r, max_a = 0, len(h)-1, 0
    while l < r:
        max_a = max(max_a, (r-l) * min(h[l], h[r]))
        if h[l] < h[r]: l += 1
        else: r -= 1
    return max_a
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Trapping Rain Water Var-63

PROBLEM STATEMENT

Compute trapped water after rainfall.

PYTHON SOLUTION

```
def trap(h: list[int]) → int:
    if not h: return 0
    l, r = 0, len(h)-1
    l_max, r_max = h[l], h[r]
    res = 0
    while l < r:
        if l_max < r_max:
            l += 1
            l_max = max(l_max, h[l])
            res += l_max - h[l]
        else:
            r -= 1
            r_max = max(r_max, h[r])
            res += r_max - h[r]
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Subarray Sum Equals K Var-64

PROBLEM STATEMENT

Find total continuous subarrays summing to k.

PYTHON SOLUTION

```
def subarraySum(nums: list[int], k: int) → int:
    count = curr = 0
    d = {0: 1}
    for n in nums:
        curr += n
        count += d.get(curr - k, 0)
        d[curr] = d.get(curr, 0) + 1
    return count
```

Time Complexity: $O(N)$ | Space Complexity: $O(N)$

Maximum Subarray (Kadane) Var-65

PROBLEM STATEMENT

Find maximum sum contiguous subarray.

PYTHON SOLUTION

```
def maxSubArray(nums: list[int]) → int:
    max_s = curr_s = nums[0]
    for n in nums[1:]:
        curr_s = max(n, curr_s + n)
        max_s = max(max_s, curr_s)
    return max_s
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Maximum Product Subarray Var-66

PROBLEM STATEMENT

Find contiguous subarray with maximum product.

PYTHON SOLUTION

```
def maxProduct(nums: list[int]) → int:
    res = max(nums)
    curMin, curMax = 1, 1
    for n in nums:
        tmp = curMax * n
        curMax = max(n * curMax, n * curMin, n)
        curMin = min(tmp, n * curMin, n)
        res = max(res, curMax)
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Product of Array Except Self Var-67

PROBLEM STATEMENT

Compute array where $\text{arr}[i]$ is product of all elements except i .

PYTHON SOLUTION

```
def productExceptSelf(nums: list[int]) → list[int]:
    res = [1] * len(nums)
    prefix = 1
    for i in range(len(nums)):
        res[i] = prefix
        prefix *= nums[i]
    postfix = 1
    for i in range(len(nums)-1, -1, -1):
        res[i] *= postfix
        postfix *= nums[i]
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Spiral Matrix Var-68

PROBLEM STATEMENT

Return elements of M x N matrix in spiral order.

PYTHON SOLUTION

```
def spiralOrder(matrix: list[list[int]]) → list[int]:
    res = []
    while matrix:
        res += matrix.pop(0)
        if matrix and matrix[0]:
            for row in matrix:
                res.append(row.pop())
        if matrix:
            res += matrix.pop()[::-1]
        if matrix and matrix[0]:
            for row in matrix[::-1]:
                res.append(row.pop(0))
    return res
```

Time Complexity: $O(M*N)$ | Space Complexity: $O(1)$

Rotate Image Var-69

PROBLEM STATEMENT

Rotate N x N matrix 90 degrees clockwise in-place.

PYTHON SOLUTION

```
def rotate(matrix: list[list[int]]) → None:
    matrix.reverse()
    for i in range(len(matrix)):
        for j in range(i):
            matrix[i][j], matrix[j][i] = matrix[j][i],
matrix[i][j]
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(1)$

First Missing Positive Var-70

PROBLEM STATEMENT

Find smallest missing positive integer.

PYTHON SOLUTION

```
def firstMissingPositive(nums: list[int]) → int:
    n = len(nums)
    for i in range(n):
        while 1 ≤ nums[i] ≤ n and nums[nums[i] - 1] ≠
nums[i]:
            nums[nums[i] - 1], nums[i] = nums[i], nums[nums[i]
- 1]
    for i in range(n):
        if nums[i] ≠ i + 1: return i + 1
    return n + 1
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

3Sum Var-71

PROBLEM STATEMENT

Find all unique triplets summing to zero.

PYTHON SOLUTION

```
def threeSum(nums: list[int]) → list[list[int]]:
    nums.sort()
    res = []
    for i in range(len(nums)-2):
        if i > 0 and nums[i] == nums[i-1]: continue
        l, r = i+1, len(nums)-1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s < 0: l += 1
            elif s > 0: r -= 1
            else:
                res.append([nums[i], nums[l], nums[r]])
                while l < r and nums[l] == nums[l+1]: l += 1
                while r > l and nums[r] == nums[r-1]: r -= 1
                l, r = l+1, r-1
    return res
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(1)$

Container With Most Water Var-72

PROBLEM STATEMENT

Find 2 lines forming container with max water volume.

PYTHON SOLUTION

```
def maxArea(h: list[int]) → int:
    l, r, max_a = 0, len(h)-1, 0
    while l < r:
        max_a = max(max_a, (r-l) * min(h[l], h[r]))
        if h[l] < h[r]: l += 1
        else: r -= 1
    return max_a
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Trapping Rain Water Var-73

PROBLEM STATEMENT

Compute trapped water after rainfall.

PYTHON SOLUTION

```
def trap(h: list[int]) → int:
    if not h: return 0
    l, r = 0, len(h)-1
    l_max, r_max = h[l], h[r]
    res = 0
    while l < r:
        if l_max < r_max:
            l += 1
            l_max = max(l_max, h[l])
            res += l_max - h[l]
        else:
            r -= 1
            r_max = max(r_max, h[r])
            res += r_max - h[r]
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Subarray Sum Equals K Var-74

PROBLEM STATEMENT

Find total continuous subarrays summing to k.

PYTHON SOLUTION

```
def subarraySum(nums: list[int], k: int) → int:
    count = curr = 0
    d = {0: 1}
    for n in nums:
        curr += n
        count += d.get(curr - k, 0)
        d[curr] = d.get(curr, 0) + 1
    return count
```

Time Complexity: $O(N)$ | Space Complexity: $O(N)$

Maximum Subarray (Kadane) Var-75

PROBLEM STATEMENT

Find maximum sum contiguous subarray.

PYTHON SOLUTION

```
def maxSubArray(nums: list[int]) → int:
    max_s = curr_s = nums[0]
    for n in nums[1:]:
        curr_s = max(n, curr_s + n)
        max_s = max(max_s, curr_s)
    return max_s
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Maximum Product Subarray Var-76

PROBLEM STATEMENT

Find contiguous subarray with maximum product.

PYTHON SOLUTION

```
def maxProduct(nums: list[int]) → int:
    res = max(nums)
    curMin, curMax = 1, 1
    for n in nums:
        tmp = curMax * n
        curMax = max(n * curMax, n * curMin, n)
        curMin = min(tmp, n * curMin, n)
        res = max(res, curMax)
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Product of Array Except Self Var-77

PROBLEM STATEMENT

Compute array where $\text{arr}[i]$ is product of all elements except i .

PYTHON SOLUTION

```
def productExceptSelf(nums: list[int]) → list[int]:
    res = [1] * len(nums)
    prefix = 1
    for i in range(len(nums)):
        res[i] = prefix
        prefix *= nums[i]
    postfix = 1
    for i in range(len(nums)-1, -1, -1):
        res[i] *= postfix
        postfix *= nums[i]
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Spiral Matrix Var-78

PROBLEM STATEMENT

Return elements of M x N matrix in spiral order.

PYTHON SOLUTION

```
def spiralOrder(matrix: list[list[int]]) → list[int]:
    res = []
    while matrix:
        res += matrix.pop(0)
        if matrix and matrix[0]:
            for row in matrix:
                res.append(row.pop())
        if matrix:
            res += matrix.pop()[::-1]
        if matrix and matrix[0]:
            for row in matrix[::-1]:
                res.append(row.pop(0))
    return res
```

Time Complexity: $O(M*N)$ | Space Complexity: $O(1)$

Rotate Image Var-79

PROBLEM STATEMENT

Rotate N x N matrix 90 degrees clockwise in-place.

PYTHON SOLUTION

```
def rotate(matrix: list[list[int]]) → None:
    matrix.reverse()
    for i in range(len(matrix)):
        for j in range(i):
            matrix[i][j], matrix[j][i] = matrix[j][i],
matrix[i][j]
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(1)$

First Missing Positive Var-80

PROBLEM STATEMENT

Find smallest missing positive integer.

PYTHON SOLUTION

```
def firstMissingPositive(nums: list[int]) → int:
    n = len(nums)
    for i in range(n):
        while 1 ≤ nums[i] ≤ n and nums[nums[i] - 1] ≠
nums[i]:
            nums[nums[i] - 1], nums[i] = nums[i], nums[nums[i]
- 1]
    for i in range(n):
        if nums[i] ≠ i + 1: return i + 1
    return n + 1
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

3Sum Var-81

PROBLEM STATEMENT

Find all unique triplets summing to zero.

PYTHON SOLUTION

```
def threeSum(nums: list[int]) → list[list[int]]:
    nums.sort()
    res = []
    for i in range(len(nums)-2):
        if i > 0 and nums[i] == nums[i-1]: continue
        l, r = i+1, len(nums)-1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s < 0: l += 1
            elif s > 0: r -= 1
            else:
                res.append([nums[i], nums[l], nums[r]])
                while l < r and nums[l] == nums[l+1]: l += 1
                while r > l and nums[r] == nums[r-1]: r -= 1
                l, r = l+1, r-1
    return res
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(1)$

Container With Most Water Var-82

PROBLEM STATEMENT

Find 2 lines forming container with max water volume.

PYTHON SOLUTION

```
def maxArea(h: list[int]) → int:
    l, r, max_a = 0, len(h)-1, 0
    while l < r:
        max_a = max(max_a, (r-l) * min(h[l], h[r]))
        if h[l] < h[r]: l += 1
        else: r -= 1
    return max_a
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Trapping Rain Water Var-83

PROBLEM STATEMENT

Compute trapped water after rainfall.

PYTHON SOLUTION

```
def trap(h: list[int]) → int:
    if not h: return 0
    l, r = 0, len(h)-1
    l_max, r_max = h[l], h[r]
    res = 0
    while l < r:
        if l_max < r_max:
            l += 1
            l_max = max(l_max, h[l])
            res += l_max - h[l]
        else:
            r -= 1
            r_max = max(r_max, h[r])
            res += r_max - h[r]
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Subarray Sum Equals K Var-84

PROBLEM STATEMENT

Find total continuous subarrays summing to k.

PYTHON SOLUTION

```
def subarraySum(nums: list[int], k: int) → int:
    count = curr = 0
    d = {0: 1}
    for n in nums:
        curr += n
        count += d.get(curr - k, 0)
        d[curr] = d.get(curr, 0) + 1
    return count
```

Time Complexity: $O(N)$ | Space Complexity: $O(N)$

Maximum Subarray (Kadane) Var-85

PROBLEM STATEMENT

Find maximum sum contiguous subarray.

PYTHON SOLUTION

```
def maxSubArray(nums: list[int]) → int:  
    max_s = curr_s = nums[0]  
    for n in nums[1:]:  
        curr_s = max(n, curr_s + n)  
        max_s = max(max_s, curr_s)  
    return max_s
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Maximum Product Subarray Var-86

PROBLEM STATEMENT

Find contiguous subarray with maximum product.

PYTHON SOLUTION

```
def maxProduct(nums: list[int]) → int:
    res = max(nums)
    curMin, curMax = 1, 1
    for n in nums:
        tmp = curMax * n
        curMax = max(n * curMax, n * curMin, n)
        curMin = min(tmp, n * curMin, n)
        res = max(res, curMax)
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Product of Array Except Self Var-87

PROBLEM STATEMENT

Compute array where $\text{arr}[i]$ is product of all elements except i .

PYTHON SOLUTION

```
def productExceptSelf(nums: list[int]) → list[int]:
    res = [1] * len(nums)
    prefix = 1
    for i in range(len(nums)):
        res[i] = prefix
        prefix *= nums[i]
    postfix = 1
    for i in range(len(nums)-1, -1, -1):
        res[i] *= postfix
        postfix *= nums[i]
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Spiral Matrix Var-88

PROBLEM STATEMENT

Return elements of M x N matrix in spiral order.

PYTHON SOLUTION

```
def spiralOrder(matrix: list[list[int]]) → list[int]:
    res = []
    while matrix:
        res += matrix.pop(0)
        if matrix and matrix[0]:
            for row in matrix:
                res.append(row.pop())
        if matrix:
            res += matrix.pop()[::-1]
        if matrix and matrix[0]:
            for row in matrix[::-1]:
                res.append(row.pop(0))
    return res
```

Time Complexity: $O(M*N)$ | Space Complexity: $O(1)$

Rotate Image Var-89

PROBLEM STATEMENT

Rotate N x N matrix 90 degrees clockwise in-place.

PYTHON SOLUTION

```
def rotate(matrix: list[list[int]]) → None:
    matrix.reverse()
    for i in range(len(matrix)):
        for j in range(i):
            matrix[i][j], matrix[j][i] = matrix[j][i],
matrix[i][j]
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(1)$

First Missing Positive Var-90

PROBLEM STATEMENT

Find smallest missing positive integer.

PYTHON SOLUTION

```
def firstMissingPositive(nums: list[int]) → int:
    n = len(nums)
    for i in range(n):
        while 1 ≤ nums[i] ≤ n and nums[nums[i] - 1] ≠
nums[i]:
            nums[nums[i] - 1], nums[i] = nums[i], nums[nums[i]
- 1]
    for i in range(n):
        if nums[i] ≠ i + 1: return i + 1
    return n + 1
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

3Sum Var-91

PROBLEM STATEMENT

Find all unique triplets summing to zero.

PYTHON SOLUTION

```
def threeSum(nums: list[int]) → list[list[int]]:
    nums.sort()
    res = []
    for i in range(len(nums)-2):
        if i > 0 and nums[i] == nums[i-1]: continue
        l, r = i+1, len(nums)-1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s < 0: l += 1
            elif s > 0: r -= 1
            else:
                res.append([nums[i], nums[l], nums[r]])
                while l < r and nums[l] == nums[l+1]: l += 1
                while r > l and nums[r] == nums[r-1]: r -= 1
                l, r = l+1, r-1
    return res
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(1)$

Container With Most Water Var-92

PROBLEM STATEMENT

Find 2 lines forming container with max water volume.

PYTHON SOLUTION

```
def maxArea(h: list[int]) → int:
    l, r, max_a = 0, len(h)-1, 0
    while l < r:
        max_a = max(max_a, (r-l) * min(h[l], h[r]))
        if h[l] < h[r]: l += 1
        else: r -= 1
    return max_a
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Trapping Rain Water Var-93

PROBLEM STATEMENT

Compute trapped water after rainfall.

PYTHON SOLUTION

```
def trap(h: list[int]) → int:
    if not h: return 0
    l, r = 0, len(h)-1
    l_max, r_max = h[l], h[r]
    res = 0
    while l < r:
        if l_max < r_max:
            l += 1
            l_max = max(l_max, h[l])
            res += l_max - h[l]
        else:
            r -= 1
            r_max = max(r_max, h[r])
            res += r_max - h[r]
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Subarray Sum Equals K Var-94

PROBLEM STATEMENT

Find total continuous subarrays summing to k.

PYTHON SOLUTION

```
def subarraySum(nums: list[int], k: int) → int:
    count = curr = 0
    d = {0: 1}
    for n in nums:
        curr += n
        count += d.get(curr - k, 0)
        d[curr] = d.get(curr, 0) + 1
    return count
```

Time Complexity: $O(N)$ | Space Complexity: $O(N)$

Maximum Subarray (Kadane) Var-95

PROBLEM STATEMENT

Find maximum sum contiguous subarray.

PYTHON SOLUTION

```
def maxSubArray(nums: list[int]) → int:
    max_s = curr_s = nums[0]
    for n in nums[1:]:
        curr_s = max(n, curr_s + n)
        max_s = max(max_s, curr_s)
    return max_s
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Maximum Product Subarray Var-96

PROBLEM STATEMENT

Find contiguous subarray with maximum product.

PYTHON SOLUTION

```
def maxProduct(nums: list[int]) → int:
    res = max(nums)
    curMin, curMax = 1, 1
    for n in nums:
        tmp = curMax * n
        curMax = max(n * curMax, n * curMin, n)
        curMin = min(tmp, n * curMin, n)
        res = max(res, curMax)
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Product of Array Except Self Var-97

PROBLEM STATEMENT

Compute array where $\text{arr}[i]$ is product of all elements except i .

PYTHON SOLUTION

```
def productExceptSelf(nums: list[int]) → list[int]:
    res = [1] * len(nums)
    prefix = 1
    for i in range(len(nums)):
        res[i] = prefix
        prefix *= nums[i]
    postfix = 1
    for i in range(len(nums)-1, -1, -1):
        res[i] *= postfix
        postfix *= nums[i]
    return res
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$

Spiral Matrix Var-98

PROBLEM STATEMENT

Return elements of M x N matrix in spiral order.

PYTHON SOLUTION

```
def spiralOrder(matrix: list[list[int]]) → list[int]:
    res = []
    while matrix:
        res += matrix.pop(0)
        if matrix and matrix[0]:
            for row in matrix:
                res.append(row.pop())
        if matrix:
            res += matrix.pop()[::-1]
        if matrix and matrix[0]:
            for row in matrix[::-1]:
                res.append(row.pop(0))
    return res
```

Time Complexity: $O(M*N)$ | Space Complexity: $O(1)$

Rotate Image Var-99

PROBLEM STATEMENT

Rotate N x N matrix 90 degrees clockwise in-place.

PYTHON SOLUTION

```
def rotate(matrix: list[list[int]]) → None:
    matrix.reverse()
    for i in range(len(matrix)):
        for j in range(i):
            matrix[i][j], matrix[j][i] = matrix[j][i],
matrix[i][j]
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(1)$

First Missing Positive Var-100

PROBLEM STATEMENT

Find smallest missing positive integer.

PYTHON SOLUTION

```
def firstMissingPositive(nums: list[int]) → int:
    n = len(nums)
    for i in range(n):
        while 1 ≤ nums[i] ≤ n and nums[nums[i] - 1] ≠
nums[i]:
            nums[nums[i] - 1], nums[i] = nums[i], nums[nums[i]
- 1]
    for i in range(n):
        if nums[i] ≠ i + 1: return i + 1
    return n + 1
```

Time Complexity: $O(N)$ | Space Complexity: $O(1)$